JMORF/MORS — Morpho-Syntax

The Structure of the Lexicon I
Lexical Types

Francis Bond
Palacký University
https://fcbond.github.io/
bond@ieee.org

Lecture 7
Location: KB-1.11

Overview

1. **Lexical Types**: why a lexical hierarchy, the seven lexical terms, defaults, a tour of the lexeme hierarchy, the Case Constraint
   Activity: all the constraints on ***give***

2. **Lexical Rules: A First Look**: how lexical rules fit into the grammar, the three kinds of rule, and two examples

Next week: lexical rules in detail, and the passive.
No problem set this week: the mid-term is due instead.

Lexical Types

Overview

- Motivation for lexical hierarchy
- Lexical terminology
- Defaults: the ' / ' notation
- Tour of the lexeme hierarchy
- The Case Constraint
- *pos* vs. *lexeme*
- In-class activity: all the constraints on ***give***

Motivation for the Lexical Hierarchy

- We've streamlined our grammar rules...
  - ...by stating some constraints as general principles
  - ...and locating lots of information in the lexicon.

- Our lexical entries currently stipulate a lot of information that is common across many entries and should be stated only once. Examples?

Motivation for the Lexical Hierarchy

- We've streamlined our grammar rules...
  - ...by stating some constraints as general principles
  - ...and locating lots of information in the lexicon.

- Our lexical entries currently stipulate a lot of information that is common across many entries and should be stated only once. Examples?
  - Every common noun: $\left[\textsc{head} \begin{bmatrix} \textit{noun} \\ \textsc{agr} \ \left[\textsc{per} \ \textit{3rd}\right] \end{bmatrix}\right]$, $\left[\textsc{spr} \ \langle \textsc{dp} \rangle\right]$, $\left[\textsc{mode} \ \textit{ref}\right]$
  - Every verb: $\left[\textsc{head} \ \textit{verb}\right]$, $\left[\textsc{mode} \ \textit{prop}\right]$, an NP subject first on ARG-ST
  - Every noun and verb: the SHAC

The Ideal Lexical Entry

- Ideally, particular lexical entries need only give phonological form, the semantic contribution, and any constraints truly idiosyncratic to the lexical entry.
  - We don't get quite that far in this book, because of our 'cutesy' role names (LOVER, GIVER, ...): the entry still has to say which argument plays which role
  - Implemented grammars such as the ERG use generic role labels (ARG1, ARG2), so the mapping from arguments to roles can live in the lexical type

Lexemes and Words (in a General Linguistic Sense)

- **Lexeme**: An abstract proto-word which gives rise to genuine words. We refer to lexemes by their 'dictionary form', e.g. 'the lexeme *run*' or 'the lexeme *dog*'.

- **Word**: A particular pairing of form and meaning. *Running* and *ran* are different words

Lexical Entries, Lexical Types & Lexical Rules

- **Lexical entries** capture the similarities among ***run, runs, running,*** and ***ran***.

- The **lexical type hierarchy** captures the similarities among ***run, sleep,*** and ***laugh***, among those and other verbs like ***devour*** and ***hand***, and among those and other words like ***book***.

  Q What do ***devour*** and ***book*** have in common?

- **Lexical rules** capture the similarities among ***runs, sleeps, devours, hands,***...

Lexical Entries, Lexical Types & Lexical Rules

- **Lexical entries** capture the similarities among ***run, runs, running,*** and ***ran***.

- The **lexical type hierarchy** captures the similarities among ***run, sleep,*** and ***laugh***, among those and other verbs like ***devour*** and ***hand***, and among those and other words like ***book***.

  Q What do ***devour*** and ***book*** have in common?
  A The SHAC: each agrees with its specifier

- **Lexical rules** capture the similarities among ***runs, sleeps, devours, hands,***...

Lexical Terminology

The book uses seven related terms

- Lexeme
- Lexical entry
- Lexical sequence
- 'Family of lexical sequences'
- Lexical rule
- Lexical rule instantiation
- Word structure

Lexical Terminology

- **Lexeme**
- **Lexical entry**
- **Lexical sequence**
- **'Family of lexical sequences'**
- Lexical rule
- Lexical rule instantiation
- Word structure

lexeme

- A type in our grammar
- The root of the sub-hierarchy of all the lexeme types we use in defining lexical entries
- So 'a lexeme' loosely refers to the type *lexeme* or one of its subtypes
- These types are generalizations about combinations of constraints
  - e.g. the lexeme type *pn-lxm* encodes the generalization that proper nouns are (usually) singular
  - the lexeme type *stv-lxm* encodes the generalization that a transitive verb takes two arguments

Lexical Entry

- A piece of the grammar that we write in the lexicon
  - e.g. 'look, we have ***book***, it's of type *cntn-lxm*, and we have ***give***, it's of type *dtv-lxm*'

$$\left\langle \text{book} \;,\; \begin{bmatrix} \textit{cntn-lxm} \\ \text{SEM} \; \begin{bmatrix} \text{INDEX} \;\; i \\ \text{RESTR} \; \left\langle \begin{bmatrix} \text{RELN} \;\; \textit{book} \\ \text{INST} \;\; i \end{bmatrix} \right\rangle \end{bmatrix} \end{bmatrix} \right\rangle \qquad \left\langle \text{give} \;,\; \begin{bmatrix} \textit{dtv-lxm} \\ \text{SEM} \; \begin{bmatrix} \text{INDEX} \;\; s \\ \text{RESTR} \; \left\langle \begin{bmatrix} \text{RELN} & \textit{give} \\ \text{SIT} & s \\ \text{GIVER} & i \\ \text{RECIPIENT} & j \\ \text{GIFT} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} \; \langle \text{X}_i, \; \text{Y}_j, \; \text{Z}_k \rangle \end{bmatrix} \right\rangle$$

Lexical Sequence

- A fully resolved feature structure that is a pair of some orthographic or phonological string with some feature structure of type *lexeme* or *word*

- There is a type *l-sequence* to encode this
  - the INPUT and OUTPUT features of lexical rules have values of type *l-sequence*

TYPE	FEATURES/CONSTRAINTS	IST
list		*feat-struc*
list(τ)	$\begin{bmatrix} \text{FIRST} & \tau \\ \text{REST} & \textit{list}(\tau) \end{bmatrix}$	*list*
l-sequence	$\begin{bmatrix} \text{FIRST} & \textit{atom} \\ \text{REST} & \langle \textit{word} \rangle \mid \langle \textit{lexeme} \rangle \end{bmatrix}$	*list*

In plain English: an *l-sequence* is a two-element list, a string followed by one *word* or *lexeme*.

Lexical Sequences?

$$\left\langle \text{book} , \begin{bmatrix} \textit{cntn-lxm} \\ \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{1} & \begin{bmatrix} \textit{3sing} \\ \text{PER} \ \ \textit{3rd} \\ \text{NUM} \ \ \textit{sg} \\ \text{GEND} \ \ \textit{neut} \end{bmatrix} \\ \text{CASE} \ \ \textit{nom} \end{bmatrix} \\ \text{VAL} & \begin{bmatrix} \text{SPR} & \langle [\text{AGR} \ \boxed{1}] \rangle \\ \text{COMPS} & \langle\rangle \\ \text{MOD} & \langle\rangle \end{bmatrix} \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} \ \ \textit{ref} \\ \text{INDEX} \ \ i \\ \text{RESTR} \ \left\langle \begin{bmatrix} \text{RELN} \ \ \textit{book} \\ \text{INST} \ \ i \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{DP}_i [\text{COUNT} \ +] \rangle \end{bmatrix} \right\rangle$$

$$\left\langle \text{book} , \begin{bmatrix} \textit{cntn-lxm} \\ \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{1} & \begin{bmatrix} \textit{plural} \\ \text{PER} \ \ \textit{3rd} \\ \text{NUM} \ \ \textit{pl} \end{bmatrix} \\ \text{CASE} \ \ \textit{acc} \end{bmatrix} \\ \text{VAL} & \begin{bmatrix} \text{SPR} & \langle [\text{AGR} \ \boxed{1}] \rangle \\ \text{COMPS} & \langle\rangle \\ \text{MOD} & \langle\rangle \end{bmatrix} \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} \ \ \textit{ref} \\ \text{INDEX} \ \ i \\ \text{RESTR} \ \left\langle \begin{bmatrix} \text{RELN} \ \ \textit{book} \\ \text{INST} \ \ i \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{DP}_i [\text{COUNT} \ +] \rangle \end{bmatrix} \right\rangle$$

Q Technically, neither of these is a lexical sequence. Why?

Lexical Sequences?

$$\left\langle \text{book} , \begin{bmatrix} \textit{cntn-lxm} \\ \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{1} & \begin{bmatrix} \textit{3sing} \\ \text{PER} \ \textit{3rd} \\ \text{NUM} \ \textit{sg} \\ \text{GEND} \ \textit{neut} \end{bmatrix} \\ \text{CASE} \ \textit{nom} \end{bmatrix} \\ \text{VAL} & \begin{bmatrix} \text{SPR} & \langle [\text{AGR} \ \boxed{1}] \rangle \\ \text{COMPS} & \langle \rangle \\ \text{MOD} & \langle \rangle \end{bmatrix} \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} \ \textit{ref} \\ \text{INDEX} \ i \\ \text{RESTR} \ \left\langle \begin{bmatrix} \text{RELN} \ \textit{book} \\ \text{INST} \ i \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{DP}_i [\text{COUNT} \ +] \rangle \end{bmatrix} \right\rangle \qquad \left\langle \text{book} , \begin{bmatrix} \textit{cntn-lxm} \\ \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} \ \boxed{1} & \begin{bmatrix} \textit{plural} \\ \text{PER} \ \textit{3rd} \\ \text{NUM} \ \textit{pl} \end{bmatrix} \\ \text{CASE} \ \textit{acc} \end{bmatrix} \\ \text{VAL} & \begin{bmatrix} \text{SPR} & \langle [\text{AGR} \ \boxed{1}] \rangle \\ \text{COMPS} & \langle \rangle \\ \text{MOD} & \langle \rangle \end{bmatrix} \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} \ \textit{ref} \\ \text{INDEX} \ i \\ \text{RESTR} \ \left\langle \begin{bmatrix} \text{RELN} \ \textit{book} \\ \text{INST} \ i \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{DP}_i [\text{COUNT} \ +] \rangle \end{bmatrix} \right\rangle$$

Q Technically, neither of these is a lexical sequence. Why?

A Lexical sequences must be **fully resolved**, and these don't fully resolve the DP on the ARG-ST and SPR lists.

‘Family of Lexical Sequences’

- A pairing of orthographic form with some underspecified feature structure of type *lexeme* or *word*, which can be fully resolved to a number of different lexical sequences
- The book often refers to lexical entries, for example, as being ‘families of lexical sequences’, because they license infinitely many fully resolved lexical sequences
- So the examples on the previous slide are technically all families of lexical sequences, because the ARG-ST and VAL constituents are not fully specified

Lexical Terminology

- Lexeme
- Lexical entry
- Lexical sequence
- 'Family of lexical sequences'
- **Lexical rule**
- **Lexical rule instantiation**
- **Word structure**

Lexical Rule

- A part of the grammar that we write that allows us to derive...
  - inflected forms of words, given lexical entries
  - or additional lexical sequences where the feature structure is some subtype of *lexeme*
    - ...which can unify with the INPUT feature of additional lexical rules until we arrive at a lexical sequence pairing a word-form with a feature structure of type *word*

Lexical Rule: An Example

3rd-Singular Verb Lexical Rule

$$\left[\begin{array}{l} \textit{i-rule} \\ \textsc{input} \quad \left\langle \boxed{3}\ ,\ \left[\begin{array}{l}\textit{verb-lxm}\\ \textsc{sem}\ \left[\textsc{restr}\ \boxed{A}\right]\end{array}\right]\right\rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{3SG}\left(\boxed{3}\right),\ \left[\begin{array}{ll} \textsc{syn} & \left[\textsc{head}\ \left[\begin{array}{ll}\textsc{form} & \textit{fin}\\ \textsc{agr} & \textit{3sing}\end{array}\right]\right] \\ \textsc{sem} & \left[\textsc{restr}\ \boxed{A} \oplus \ldots\right] \\ \textsc{arg-st} & \left\langle\left[\textsc{case}\ \textit{nom}\right],\ \ldots\right\rangle \end{array}\right]\right\rangle \end{array}\right]$$

In plain English: take a verb lexeme, add the 3rd singular ending, and make it a finite word with a nominative subject that agrees with it.

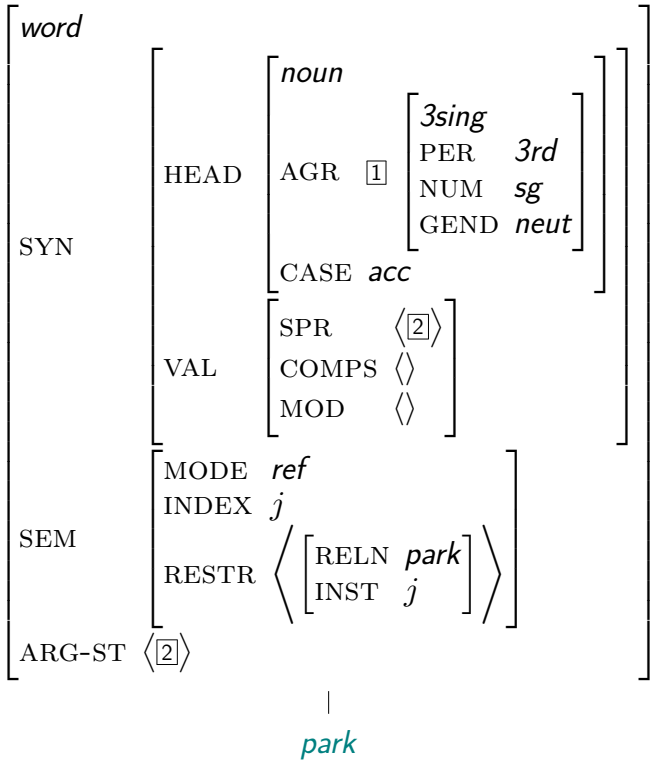
Lexical Rule Instantiation

- A fully resolved feature structure that corresponds to one of those rules in the grammar

- That is, the rule with a particular lexical sequence as its INPUT and the corresponding lexical sequence as its OUTPUT
  - e.g. the 3rd-Singular Verb Lexical Rule with ⟨sleep, ...⟩ as INPUT and ⟨sleeps, ...⟩ as OUTPUT
  - one rule in the grammar; one instantiation per word it licenses

Word Structure

- The bottom part of the tree
- A fully resolved feature structure of type *word* over some string which is the spelling or pronunciation of the word
- e.g. *park* in *the cat sleeps in the park*



Defaults: the ‘ / ’ Notation

- Generalizations with exceptions are common:
  - Most nouns in English aren't marked for CASE, but pronouns are.
  - Most proper nouns are singular, but some (***the Alps***, sports teams) are plural.
  - Most verbs take an NP subject first on ARG-ST, but a few (Chapter 11) do not.

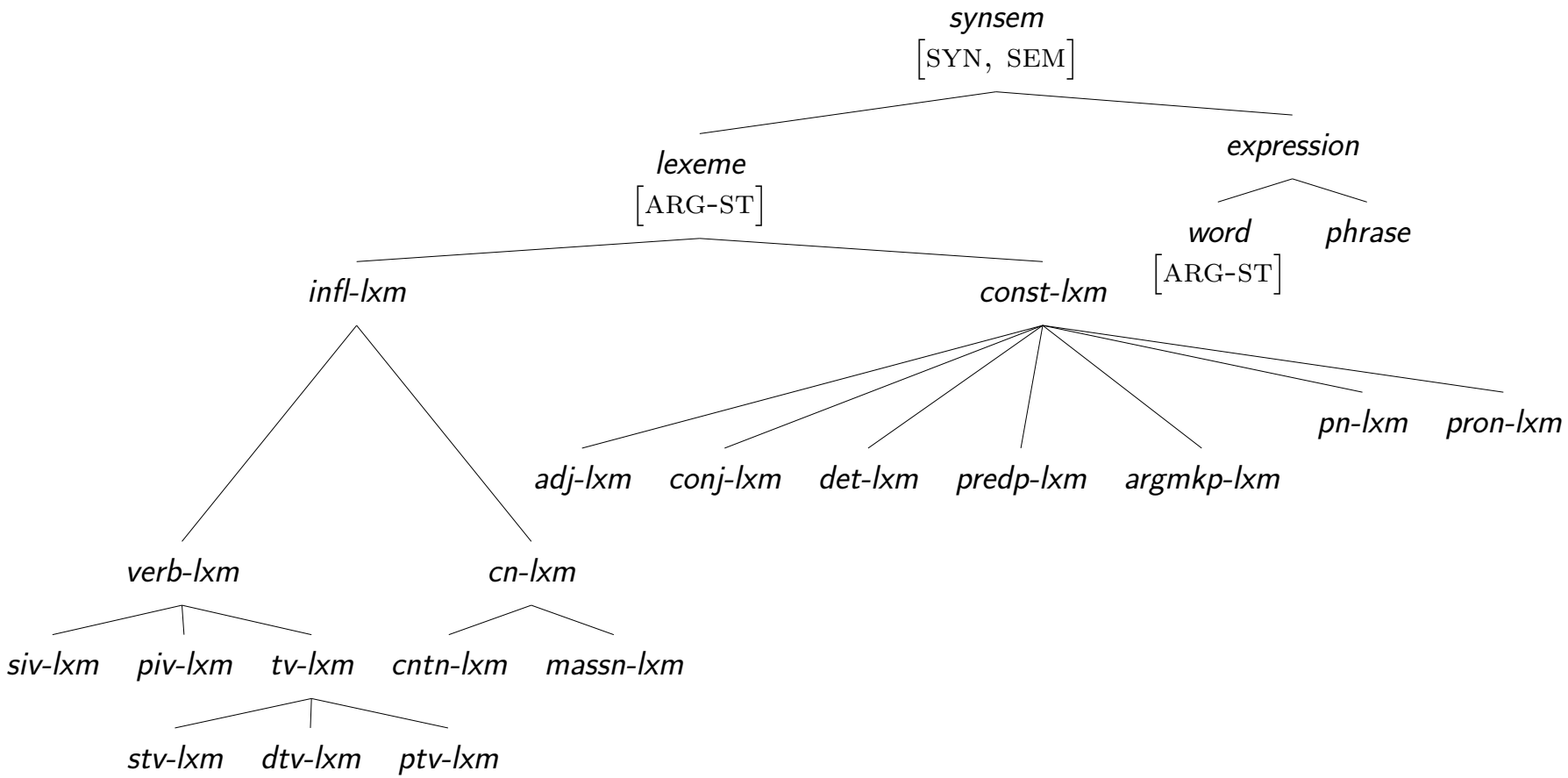
- A constraint written with a slash, e.g. $\left[\text{NUM} \;/\; sg\right]$, is a **default**: it holds unless a subtype or a lexical entry says otherwise.

- A constraint without a slash is **inviolable**: a subtype that contradicts it can have no instances.

- Defaults are all resolved in the lexicon: a *word* in a tree never has a ‘ / ’ in it.

- More on how defaults combine next week.

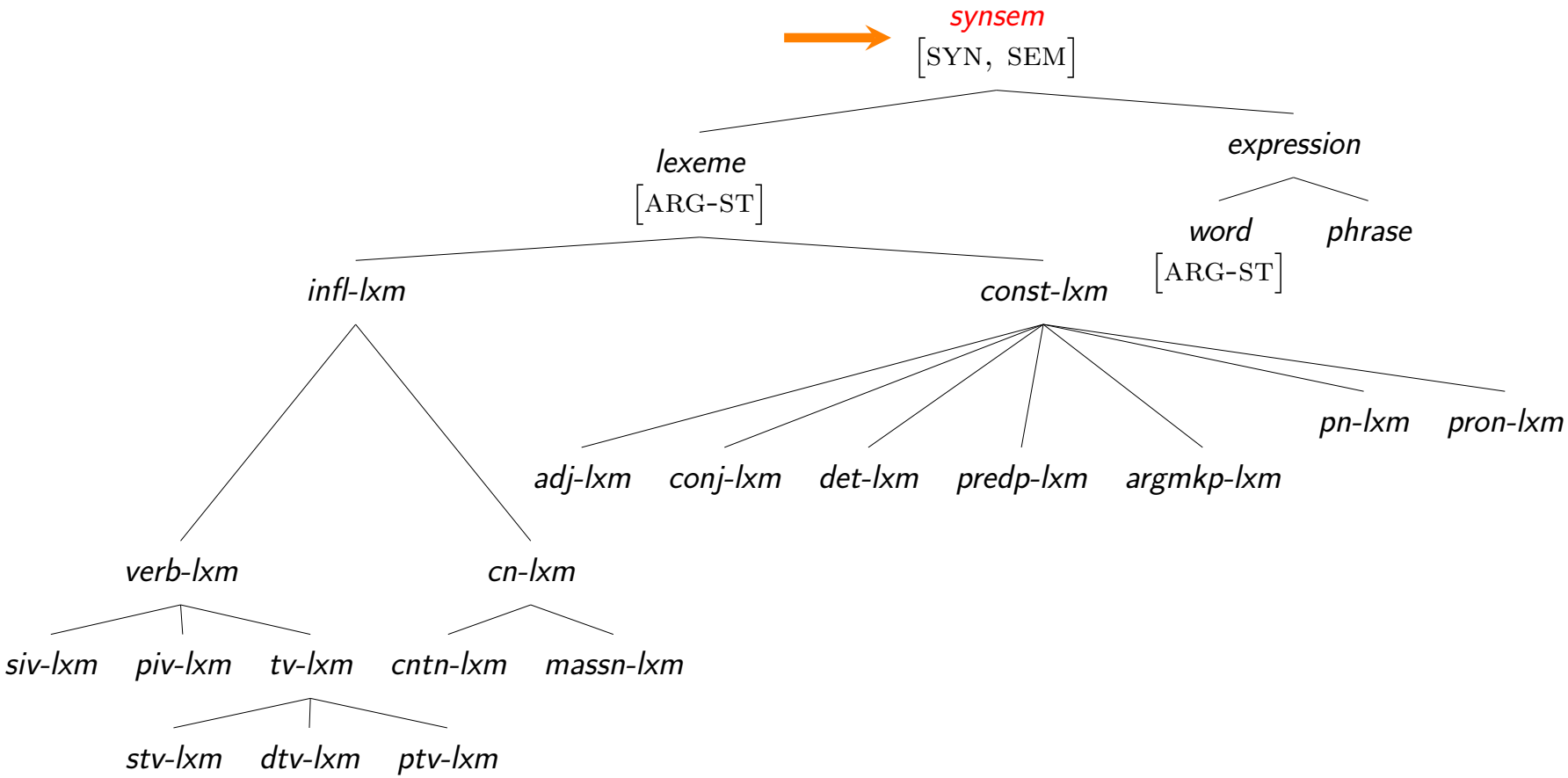
Our Lexeme Hierarchy (for English)



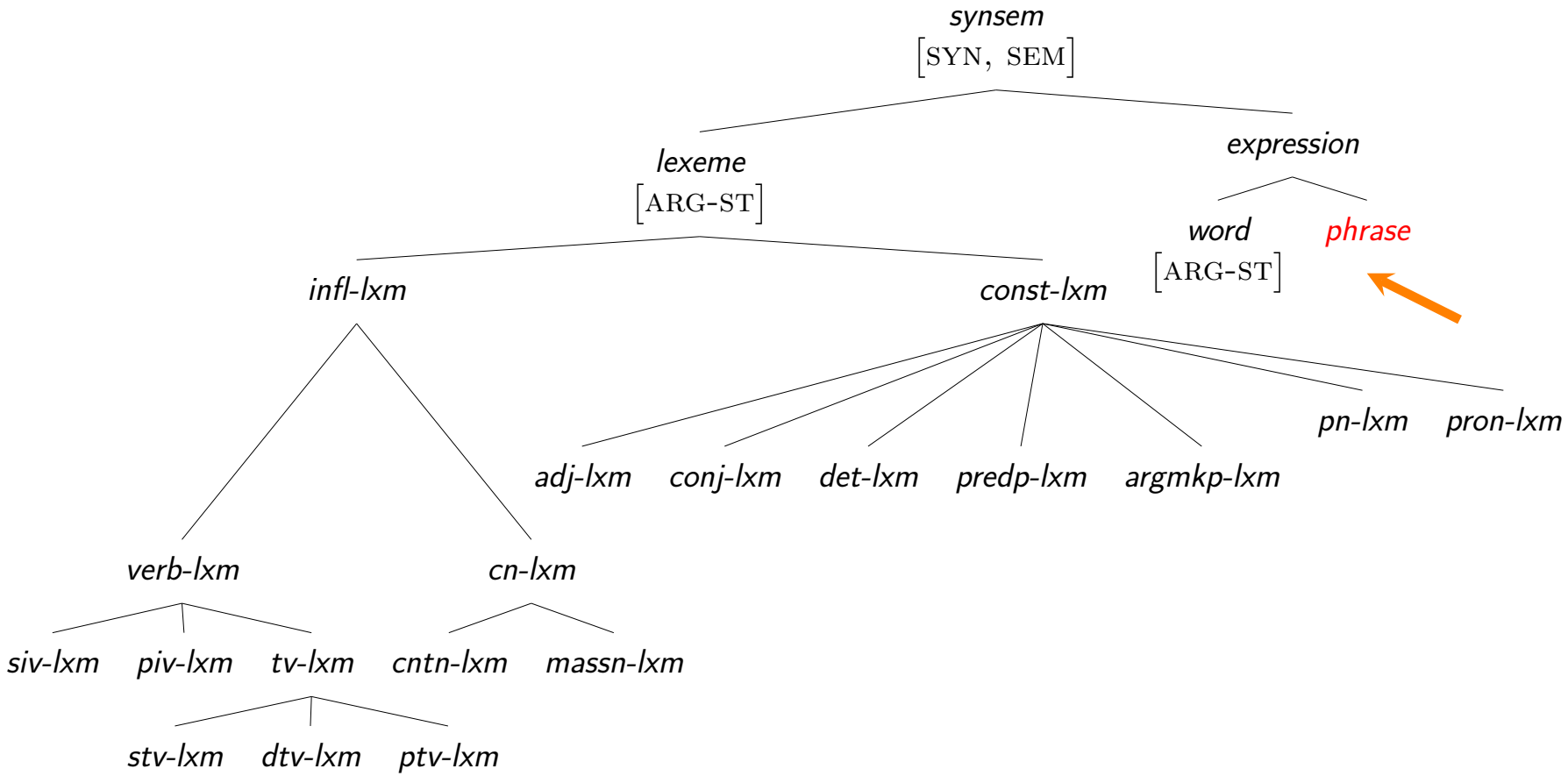
Functions of Types

- Stating what features are appropriate for what categories
- Stating generalizations
  - Constraints that apply to (almost) all instances
  - Generalizations about selection — where instances of that type can appear

Every *synsem* has the Features SYN and SEM

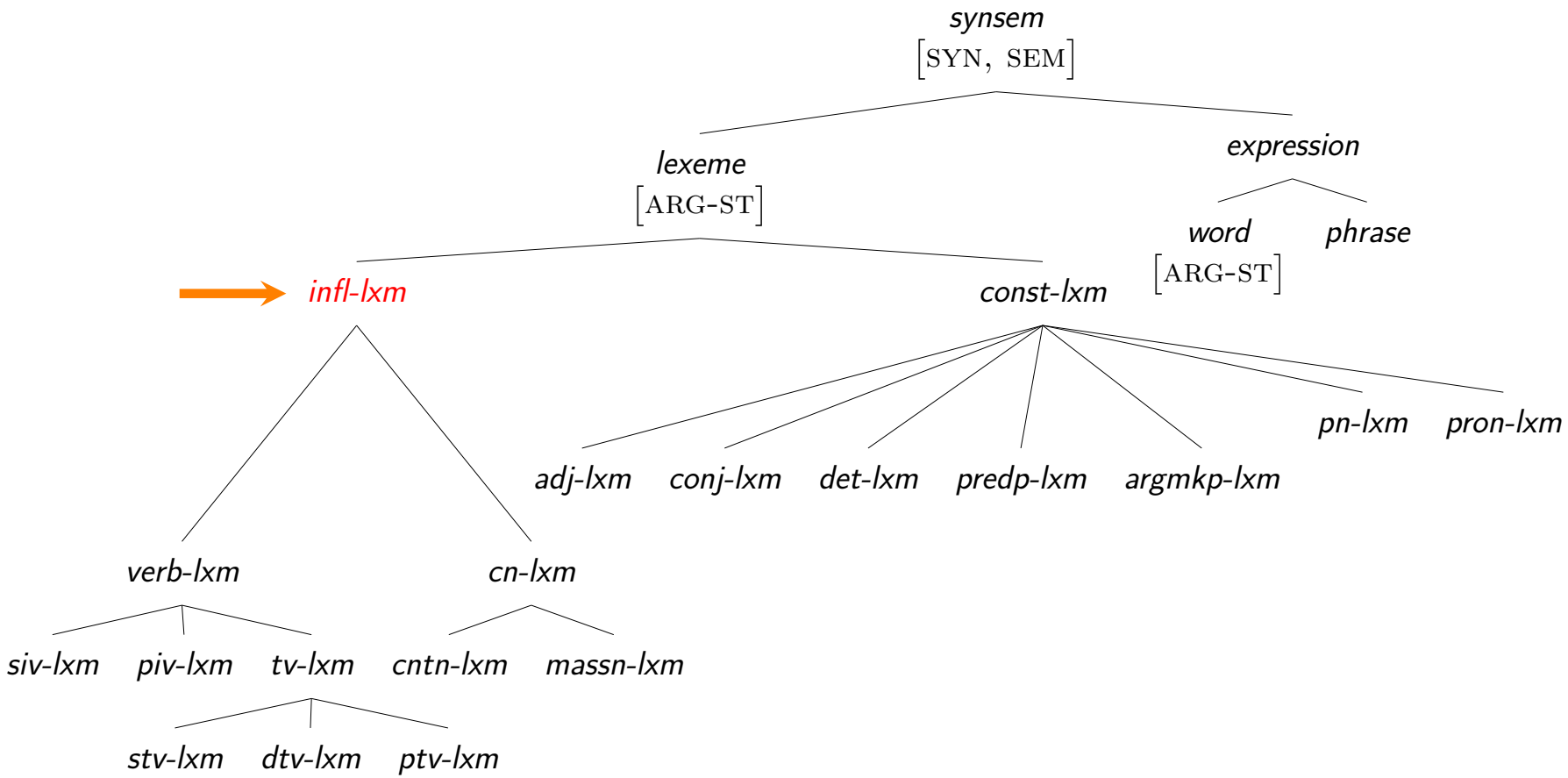


No ARG-ST on *phrase*



ARG-ST is declared on *lexeme* and on *word*, so only lexical items have it; a phrase never does.

A Constraint on *infl-lxm*: the SHAC

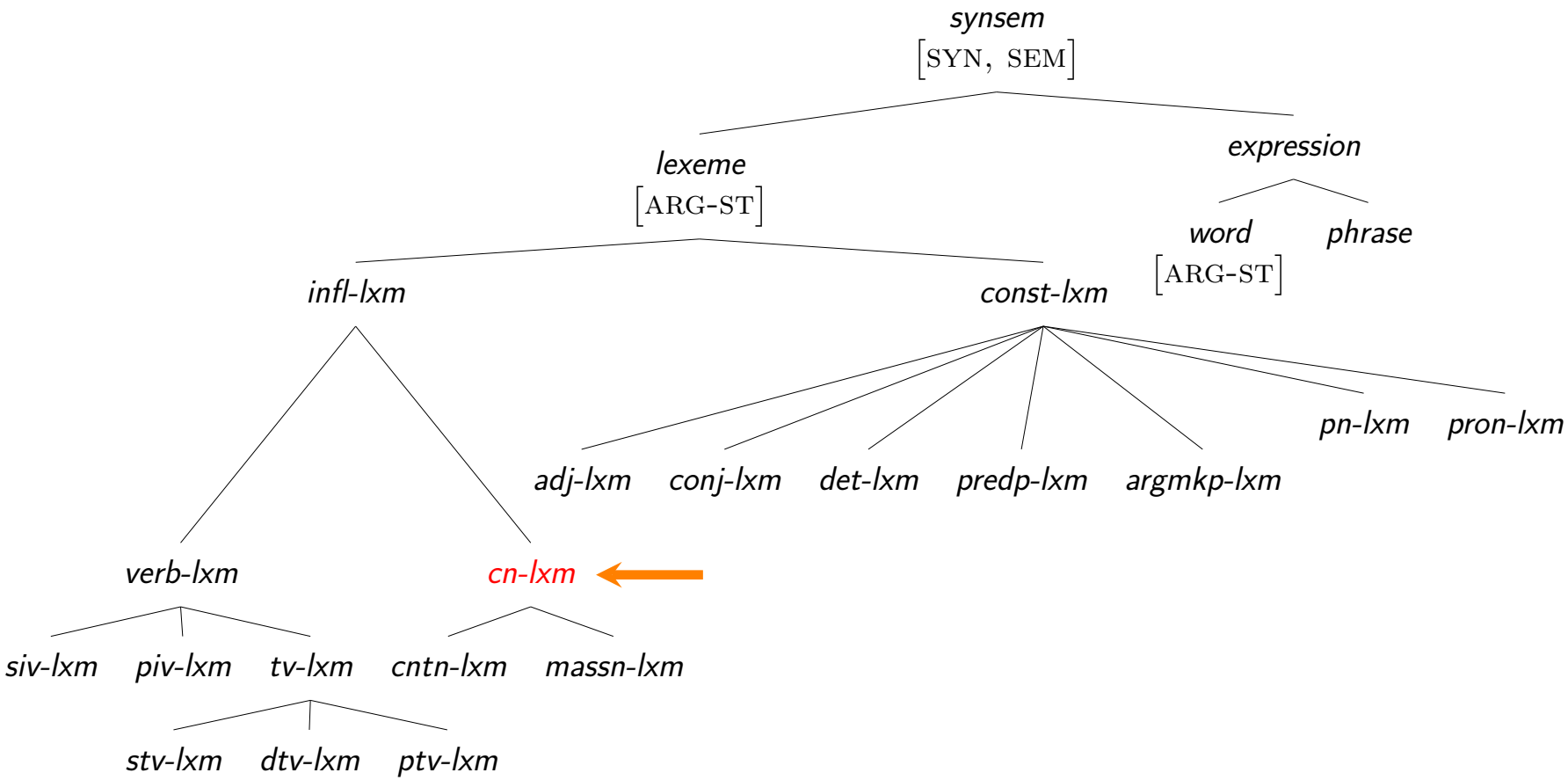


A Constraint on *infl-lxm*: the SHAC

$$\textit{infl-lxm}: \begin{bmatrix} \text{SYN} & \begin{bmatrix} \text{VAL} & \left[\text{SPR} \left\langle \left[\text{AGR } \boxed{1}\right] \right\rangle\right] \\ \text{HEAD} & \left[\text{AGR } \boxed{1}\right] \end{bmatrix} \end{bmatrix}$$

In plain English: every inflecting lexeme (nouns and verbs) has the same AGR value as its specifier. Stated once here, it is inherited by every noun and verb in the lexicon.

Constraints on *cn-lxm*

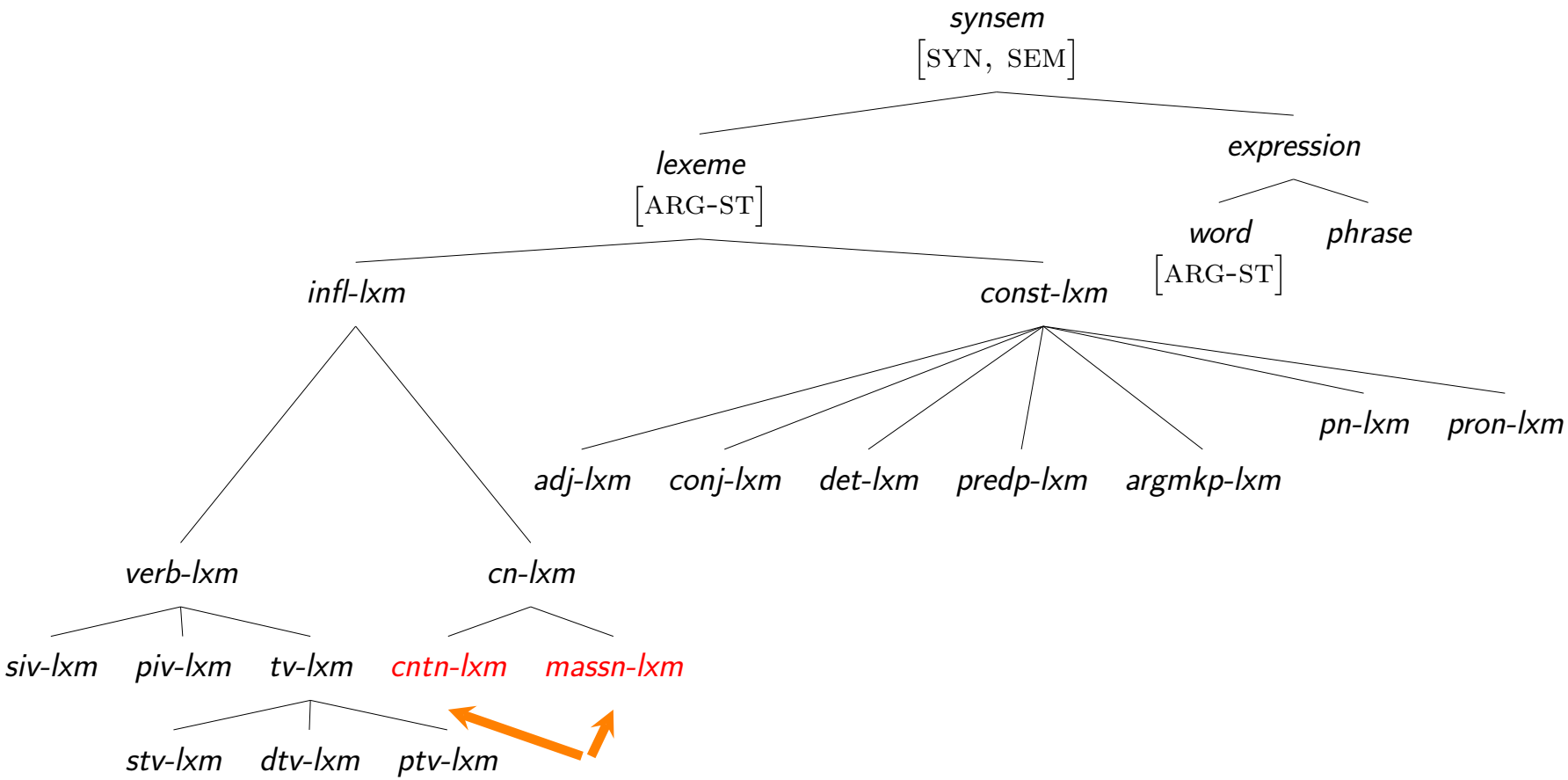


Constraints on *cn-lxm*

cn-lxm:
$$\begin{bmatrix} \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{noun} \\ \text{AGR} & \begin{bmatrix} \text{PER} & \textit{3rd} \end{bmatrix} \end{bmatrix} \\ \text{VAL} & \begin{bmatrix} \text{SPR} & \left\langle \begin{bmatrix} \text{HEAD} & \textit{det} \\ \text{INDEX} & \textit{i} \end{bmatrix} \right\rangle \end{bmatrix} \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} & / \ \textit{ref} \\ \text{INDEX} & \textit{i} \end{bmatrix} \\ \text{ARG-ST} & \langle \text{X} \rangle \oplus \ / \ \langle\rangle \end{bmatrix}$$

In plain English: a common noun is 3rd person, takes a determiner as its specifier, shares its index with that determiner, is normally referential, and normally has no arguments beyond the specifier.

Formally Distinguishing Count vs. Mass Nouns



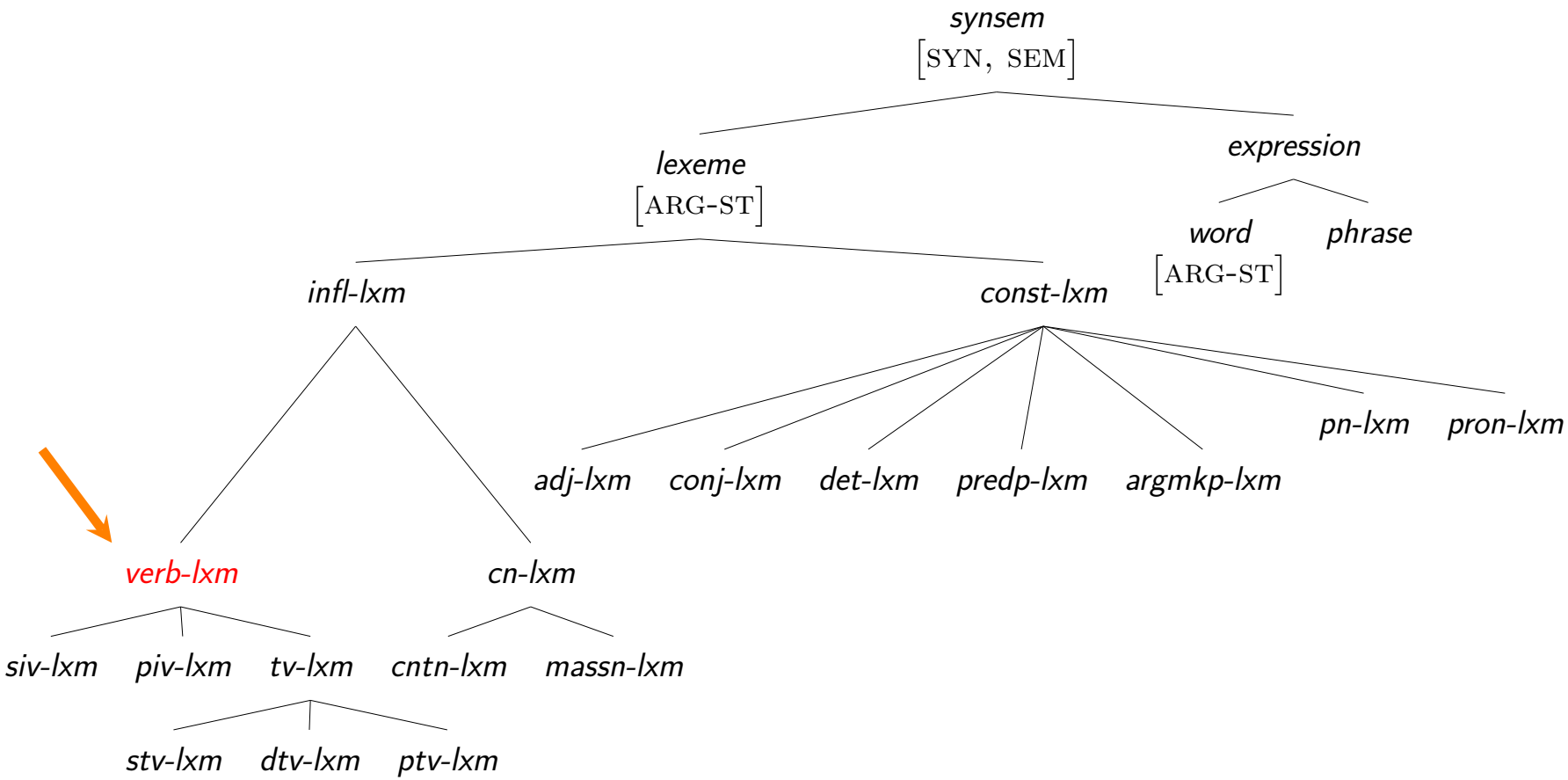
Formally Distinguishing Count vs. Mass Nouns

cntn-lxm: $\left[\text{SYN}\ \left[\text{VAL}\ \left[\text{SPR}\ \left\langle\left[\text{COUNT}\ +\right]\right\rangle\right]\right]\right]$

massn-lxm: $\left[\text{SYN}\ \left[\text{VAL}\ \left[\text{SPR}\ \left\langle\left[\text{COUNT}\ -\right]\right\rangle\right]\right]\right]$

The only difference is which determiners they take (*a book*, *much furniture*)

Constraints on *verb-lxm*

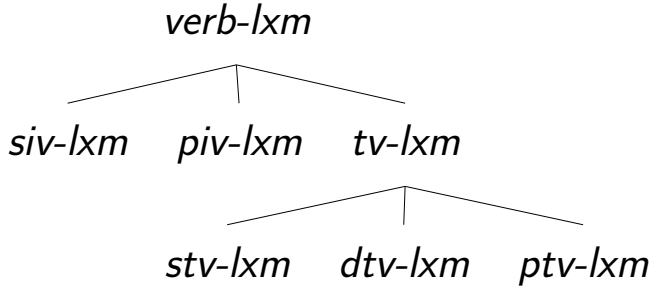


Constraints on *verb-lxm*

$$\textit{verb-lxm}: \begin{bmatrix} \textsc{syn} & \left[\textsc{head}\ \textit{verb}\right] \\ \textsc{sem} & \left[\textsc{mode}\ \textit{prop}\right] \\ \textsc{arg-st} & / \left\langle \textsc{np},\ \ldots \right\rangle \end{bmatrix}$$

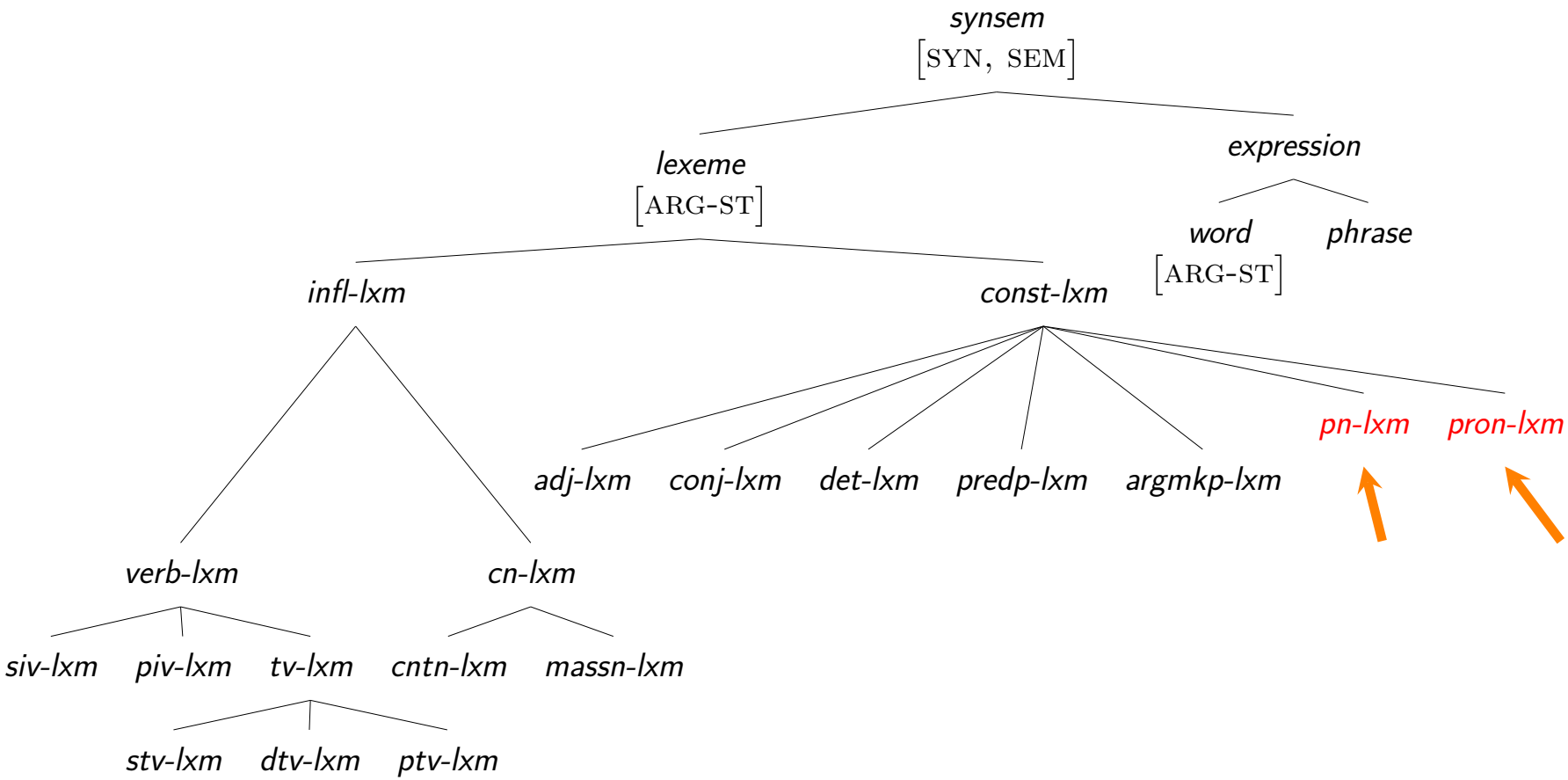
In plain English: a verb lexeme has HEAD *verb*, expresses a proposition, and by default its first argument (its subject) is an NP. The default lets a few verbs take other subjects (*That Kim left surprised me*).

Subtypes of *verb-lxm*



- *verb-lxm*: $\left[\textsc{arg-st}\ /\left\langle \mathrm{NP},\ \ldots\right\rangle\right]$
  - *siv-lxm*: $\left[\textsc{arg-st}\ \left\langle \mathrm{NP}\right\rangle\right]$ (strict intransitive: *sleep*)
  - *piv-lxm*: $\left[\textsc{arg-st}\ \left\langle \mathrm{NP},\ \mathrm{PP}\right\rangle\right]$ (prepositional intransitive: *rely*)
  - *tv-lxm*: $\left[\textsc{arg-st}\ \left\langle \mathrm{NP},\ \mathrm{NP},\ \ldots\right\rangle\right]$ (transitive)
    - *stv-lxm*: $\left[\textsc{arg-st}\ \left\langle \mathrm{NP},\ \mathrm{NP}\right\rangle\right]$ (strict transitive: *devour*)
    - *dtv-lxm*: $\left[\textsc{arg-st}\ \left\langle \mathrm{NP},\ \mathrm{NP},\ \mathrm{NP}\right\rangle\right]$ (ditransitive: *hand*)
    - *ptv-lxm*: $\left[\textsc{arg-st}\ \left\langle \mathrm{NP},\ \mathrm{NP},\ \mathrm{PP}\right\rangle\right]$ (prepositional transitive: *put*)

Proper Nouns and Pronouns



Proper Nouns and Pronouns

$$\textit{pn-lxm}: \begin{bmatrix} \textsc{syn} & \begin{bmatrix} \textsc{head} & \begin{bmatrix} \textit{noun} \\ \textsc{agr} & \begin{bmatrix} \textsc{per} & \textit{3rd} \\ \textsc{num} & \textit{/ sg} \end{bmatrix} \end{bmatrix} \end{bmatrix} \\ \textsc{sem} & \begin{bmatrix} \textsc{mode} & \textit{ref} \end{bmatrix} \\ \textsc{arg-st} & \textit{/} \langle\rangle \end{bmatrix}$$

$$\textit{pron-lxm}: \begin{bmatrix} \textsc{syn} & \begin{bmatrix} \textsc{head} & \textit{noun} \end{bmatrix} \\ \textsc{sem} & \begin{bmatrix} \textsc{mode} & \textit{/ ref} \end{bmatrix} \\ \textsc{arg-st} & \langle\rangle \end{bmatrix}$$

In plain English: proper nouns are 3rd person, usually singular (but *the Alps*) and usually take no specifier (but *the Kim I know*). Pronouns are nouns with no arguments, usually referential (but *itself* is $\begin{bmatrix} \textsc{mode} & \textit{ana} \end{bmatrix}$).

The Case Constraint

An outranked NP is $\left[\textsc{case}\ \textit{acc}\right]$.

Which of these NPs does it make accusative?

- object of verb
- second object of verb
- object of argument-marking preposition
- object of predicational preposition

The Case Constraint

An outranked NP is $\left[\text{CASE } \textit{acc}\right]$.

Which of these NPs does it make accusative?

- object of verb ✓
- second object of verb ✓
- object of argument-marking preposition ✓
- object of predicational preposition (✓)

Everything after the first element of an ARG-ST list is outranked. The object of an argument-marking preposition has the same rank as its PP, so it is outranked by the verb's subject too; the object of a predicational preposition is outranked within the preposition's own ARG-ST.

The Case Constraint, Continued

An outranked NP is $\left[\text{CASE } \textit{acc}\right]$.

- Subjects of verbs
  - Should we add a clause to cover nominative subjects, e.g. un-outranked NPs are $\left[\text{CASE } \textit{nom}\right]$?
- Any other instances of case marking in English?
- Does it apply to case systems in other languages?

The Case Constraint, Continued

An outranked NP is $\left[\text{CASE } \textit{acc}\right]$.

- Subjects of verbs
  - Should we add a clause to cover nominative subjects, e.g. un-outranked NPs are $\left[\text{CASE } \textit{nom}\right]$?
    * No. *We expect them to leave.* (Chapter 12)
      them is the SPR of ***leave*** and a COMPS member of ***expect***, so this would conflict
    * Lexical rules for finite verbs will handle nominative subjects.

- Any other instances of case marking in English?
  Possessives: *my*, *Kim's* (Chapter 6, Problem 4).

- Does it apply to case systems in other languages?
  No: the Case Constraint is an English-specific constraint.

Apparent Redundancy of *pos* and *lexeme*

- Why do we need both the *pos* subhierarchy and lexeme types?

- *pos*:
  - Applies to words and phrases; models the relationship between them
    e.g. 'verbs head VPs', 'nouns head NPs'
  - Constrains which features are appropriate (no AUX on *noun*)

- *lexeme*:
  - Generalizations about combinations of constraints

Lexical Types & Lexical Rules

- Lexical entries capture the similarities among ***run, runs, running,*** and ***ran***.

- The lexical type hierarchy captures the similarities among ***run, sleep,*** and ***laugh***, among those and other verbs like ***devour*** and ***hand***, and among those and other words like ***book***.

  – What do ***devour*** and ***book*** have in common? The SHAC

- Lexical rules capture the similarities among ***runs, sleeps, devours, hands,***...

Activity: All the Constraints on *give*

Here is the lexical entry for ***give***. Which types does it inherit from, and what does each one add?

$$\left\langle \text{give} \ , \begin{bmatrix} \textit{dtv-lxm} & \\ \text{SEM} & \begin{bmatrix} \text{INDEX} & s \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{give} \\ \text{SIT} & s \\ \text{GIVER} & i \\ \text{RECIPIENT} & j \\ \text{GIFT} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{X}_i, \ \text{Y}_j, \ \text{Z}_k \rangle \end{bmatrix} \right\rangle$$

Activity: All the Constraints on *give*

Here is the lexical entry for ***give***. Which types does it inherit from, and what does each one add?

$$\left\langle \text{give} \ , \begin{bmatrix} \textit{dtv-lxm} & \\ \text{SEM} & \begin{bmatrix} \text{INDEX} & s \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{give} \\ \text{SIT} & s \\ \text{GIVER} & i \\ \text{RECIPIENT} & j \\ \text{GIFT} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{X}_i, \ \text{Y}_j, \ \text{Z}_k \rangle \end{bmatrix} \right\rangle$$

- *dtv-lxm* $\subset$ *tv-lxm*: $\left[\text{ARG-ST} \ \langle \text{NP}, \ \text{NP}, \ \text{NP} \rangle\right]$
- *verb-lxm*: $\left[\text{HEAD} \ \textit{verb}\right]$, $\left[\text{MODE} \ \textit{prop}\right]$, $\left[\text{ARG-ST} \ / \langle \text{NP}, \ \ldots \rangle\right]$
- *infl-lxm*: the SHAC
- *lexeme*, *synsem*: the features ARG-ST, SYN and SEM are appropriate

Step 1: *dtv-lxm* and *tv-lxm*

$$\left\langle \text{give} \ , \begin{bmatrix} \textit{dtv-lxm} & & \\ \text{SEM} & \begin{bmatrix} \text{INDEX} & s & \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{give} \\ \text{SIT} & s \\ \text{GIVER} & i \\ \text{RECIPIENT} & j \\ \text{GIFT} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{NP}_i, \text{NP}_j, \text{NP}_k \rangle & \end{bmatrix} \right\rangle$$

dtv-lxm says the three arguments are all NPs: the three variables X, Y, Z are resolved to NP.

Step 2: *verb-lxm*

$$\left\langle \text{give} \ , \begin{bmatrix} \textit{dtv-lxm} & \\ \text{SYN} & \left[\text{HEAD} \ \textit{verb}\right] \\ \text{SEM} & \begin{bmatrix} \text{MODE} & \textit{prop} \\ \text{INDEX} & s \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{give} \\ \text{SIT} & s \\ \text{GIVER} & i \\ \text{RECIPIENT} & j \\ \text{GIFT} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{NP}_i, \ \text{NP}_j, \ \text{NP}_k \rangle \end{bmatrix} \right\rangle$$

verb-lxm adds $\left[\text{HEAD} \ \textit{verb}\right]$ and $\left[\text{MODE} \ \textit{prop}\right]$. Its default $\left[\text{ARG-ST} \ / \left\langle \text{NP}, \ \ldots \right\rangle\right]$ is already satisfied, so it adds nothing new.

Step 3: *infl-lxm* (the SHAC)

$$\left\langle \text{give} \ , \begin{bmatrix} \textit{dtv-lxm} & & \\ \text{SYN} & \begin{bmatrix} \text{HEAD} & \begin{bmatrix} \textit{verb} \\ \text{AGR} \ \boxed{1} \end{bmatrix} \\ \text{VAL} & \left[\text{SPR} \ \left\langle \left[\text{AGR} \ \boxed{1}\right] \right\rangle\right] \end{bmatrix} \\ \text{SEM} & \begin{bmatrix} \text{MODE} & \textit{prop} \\ \text{INDEX} & s \\ \text{RESTR} & \left\langle \begin{bmatrix} \text{RELN} & \textit{give} \\ \text{SIT} & s \\ \text{GIVER} & i \\ \text{RECIPIENT} & j \\ \text{GIFT} & k \end{bmatrix} \right\rangle \end{bmatrix} \\ \text{ARG-ST} & \langle \text{NP}_i, \ \text{NP}_j, \ \text{NP}_k \rangle \end{bmatrix} \right\rangle$$

The SHAC identifies the verb's AGR with its specifier's. Nothing yet links SPR to ARG-ST: the ARP is a constraint on *word*, so it applies only after a lexical rule.

Activity: All the Constraints on *give*

$$\left\langle \text{give} \;,\; \left[\begin{array}{ll} \textit{dtv-lxm} & \\ \text{SYN} & \left[\begin{array}{ll} \text{HEAD} & \left[\begin{array}{l} \textit{verb} \\ \text{AGR}\ \boxed{1} \end{array}\right] \\ \text{VAL} & \left[\text{SPR}\ \left\langle \left[\text{AGR}\ \boxed{1}\right] \right\rangle\right] \end{array}\right] \\ \text{SEM} & \left[\begin{array}{ll} \text{MODE} & \textit{prop} \\ \text{INDEX} & s \\ \text{RESTR} & \left\langle \left[\begin{array}{ll} \text{RELN} & \textit{give} \\ \text{SIT} & s \\ \text{GIVER} & i \\ \text{RECIPIENT} & j \\ \text{GIFT} & k \end{array}\right] \right\rangle \end{array}\right] \\ \text{ARG-ST} & \left\langle \text{NP}_i,\ \text{NP}_j,\ \text{NP}_k \right\rangle \end{array}\right] \right\rangle$$

Compare the entry we started from: HEAD, VAL, MODE and the NP types all came from the hierarchy. The entry itself only had to say the form, the meaning and the type.

Lexical Rules: A First Look

Overview

- How lexical rules fit in: tree structures, lexical sequences, lexical licensing
- Three kinds of lexical rule: inflectional, derivational, post-inflectional
- Two examples: the Plural Noun LR and the Agent Nominalization LR
- Advice on writing lexical rules

How Lexical Rules Fit In: Well-Formed Tree Structure

Φ is a well-formed tree structure according to grammar G if and only if:

1. Φ is a tree structure,

2. the label of Φ's root node satisfies the constraint $\left[\begin{array}{l}\text{SYN} \\ \end{array}\right.\left[\begin{array}{ll}\text{HEAD} & \left[\begin{array}{ll}\textit{verb} & \\ \text{FORM} & \textit{fin}\end{array}\right] \\ \text{VAL} & \left[\begin{array}{ll}\text{COMPS} & \langle\rangle \\ \text{SPR} & \langle\rangle\end{array}\right]\end{array}\left.\right]$

   and

3. each local subtree within Φ is either phrasally licensed or lexically licensed.

A sentence is a finite clause with nothing missing, where every branching is licensed by a rule and every leaf by the lexicon.

How Lexical Rules Fit In: Lexical Sequences

$\langle \omega, \phi \rangle$ is a lexical sequence if and only if ω is a phonological form (an atom), ϕ is a feature structure, and either:

1. G contains some lexical entry $\langle d_1, d_2 \rangle$ such that ω satisfies d_1 and ϕ satisfies d_2, or

2. there is some lexical rule instantiation licensed by G (a feature structure of type *l-rule*) whose OUTPUT value is $\langle \omega, \phi \rangle$.

A form paired with a feature structure is a lexical sequence if it comes straight from a lexical entry, or is the output of a lexical rule. The second clause is recursive: the rule's input must itself be a lexical sequence, so rules can feed each other.

How Lexical Rules Fit In: Lexical Licensing

A word structure of the form on the right is licensed if and only if:

1. $\langle \omega, \phi \rangle$ is a lexical sequence, where ϕ is of type *word*,
2. (Case Constraint:) an outranked NP is [CASE acc], and
3. ϕ satisfies the Binding Theory.

$$\begin{array}{c} \phi \\ | \\ \omega \end{array}$$

Only lexical sequences of type *word* can appear in trees. Lexemes have to pass through at least one lexical rule to become words: that is what the inflectional rules are for.

Parsimony & Plausibility

- Lexical rules capture **productive** generalizations.
  - When a new word enters the lexicon, it is assigned a lexical type, and we expect every lexical rule that applies to that type to apply to the new word too.
- There may be some 'precompiling' going on as well.
  - Psycholinguistically, very frequent words may just be stored ready-made rather than derived each time.

Three Kinds of Lexical Rules (very English-specific!)

- **Inflectional**: *lexeme* to *word* (takes us 'over the boundary' from lexeme to word)

  Examples?

- **Derivational**: *lexeme* to *lexeme*

  Examples?

- **Post-Inflectional**: *word* to *word*
  (Chapters 11, 13, 14)

Three Kinds of Lexical Rules (very English-specific!)

- **Inflectional**: *lexeme* to *word* (takes us 'over the boundary' from lexeme to word)

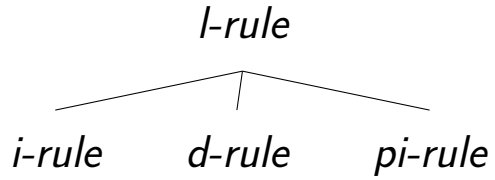
  Examples?
  Singular Noun LR, Plural Noun LR, 3rd-Singular Verb LR, Non-3rd-Singular Verb LR, Past-Tense Verb LR, Base Form LR, Constant Lexeme LR

- **Derivational**: *lexeme* to *lexeme*

  Examples?
  Present Participle LR, Past Participle LR, Agent Nominalization LR

- **Post-Inflectional**: *word* to *word*
  (Chapters 11, 13, 14)

Three Subtypes of *l-rule*



$$\textit{l-rule}: \begin{bmatrix} \textsc{input} & \textit{l-sequence} \left\langle \textsc{x} \ , \begin{bmatrix} \textsc{sem} \ / \boxed{2} \end{bmatrix} \right\rangle \\ \textsc{output} & \textit{l-sequence} \left\langle \textsc{y} \ , \begin{bmatrix} \textsc{sem} \ / \boxed{2} \end{bmatrix} \right\rangle \end{bmatrix}$$

$$\textit{i-rule}: \begin{bmatrix} \textsc{input} & \left\langle \textsc{x} \ , \begin{bmatrix} \textit{lexeme} \\ \textsc{syn} \quad \boxed{3} \\ \textsc{arg-st} \quad \boxed{A} \end{bmatrix} \right\rangle \\ \textsc{output} & \left\langle \textsc{y} \ , \begin{bmatrix} \textit{word} \\ \textsc{syn} \quad \boxed{3} \\ \textsc{arg-st} \quad \boxed{A} \end{bmatrix} \right\rangle \end{bmatrix} \qquad \textit{d-rule}: \begin{bmatrix} \textsc{input} & \left\langle \textsc{x} \ , \begin{bmatrix} \textit{lexeme} \\ \textsc{syn} \ / \boxed{3} \end{bmatrix} \right\rangle \\ \textsc{output} & \left\langle \textsc{y} \ , \begin{bmatrix} \textit{lexeme} \\ \textsc{syn} \ / \boxed{3} \end{bmatrix} \right\rangle \end{bmatrix}$$

l-rule: the output keeps the input's meaning, by default. *i-rule*: the output is a *word* with the same SYN and ARG-ST, no exceptions. *d-rule*: the output is another *lexeme* with the same SYN, by default.

Plural Noun LR

$$\left[\begin{array}{l} \textit{i-rule} \\ \textsc{input} \quad \langle \boxed{1} \ , \ \textit{cntn-lxm} \rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{NPL}(\boxed{1}), \ \left[\begin{array}{l} \textit{word} \\ \textsc{syn} \ \left[\textsc{head} \ \left[\textsc{agr} \ \left[\textsc{num} \ \textit{pl}\right]\right]\right] \end{array}\right] \right\rangle \end{array}\right]$$

Take any count noun lexeme $\boxed{1}$, add the plural ending to its form, and make it a word whose number is plural. That is all the rule itself says; everything else comes from the types.

A Derivational Rule: Agent Nominalization

$$\left[\begin{array}{l} \textit{d-rule} \\ \textsc{input} \quad \left\langle \boxed{2}\ , \left[\begin{array}{ll} \textit{verb-lxm} & \\ \textsc{sem} & \left[\textsc{index}\ s\right] \\ \textsc{arg-st} & \left\langle \mathrm{X}_i\ ,\ \mathrm{NP}_j \right\rangle \end{array}\right] \right\rangle \\ \textsc{output} \quad \left\langle \mathrm{F}_{\textit{-er}}\left(\boxed{2}\right), \left[\begin{array}{ll} \textit{cntn-lxm} & \\ \textsc{sem} & \left[\textsc{index}\ i\right] \\ \textsc{arg-st} & \left\langle \mathrm{Y}\ \left(,\ \mathrm{PP}_j\left[\textsc{form}\ \textit{of}\right]\right)\right\rangle \end{array}\right] \right\rangle \end{array}\right]$$

Add *-er* to a strictly transitive verb lexeme. The result is a count noun lexeme whose index is the verb's subject index i: a ***driver*** is the one who drives. The verb's object j can only appear as an *of*-PP: *a builder of bridges*.

Practicalities — Writing Lexical Rules

- Determine the type of the LR.
- Determine the class of possible inputs.
- Determine what should change.
  - If INPUT and OUTPUT values are identified (by default or otherwise) and only OUTPUT value is mentioned, then...
    information is added. e.g. Plural Noun LR

    (Lexical sequences incompatible with that value are not possible inputs)
  - If INPUT and OUTPUT values are identified by default, but different values are given on the INPUT and OUTPUT of the rule, then...
    information is changed. e.g. Agent Nominalization LR
  - If INPUT and OUTPUT values are identified by an inviolable constraint, but different values are given on the INPUT and OUTPUT of the rule, then...
    there is no well-formed input: the rule can never apply.

Summary

- Lexical entries say only what is idiosyncratic; the **lexeme hierarchy** supplies the rest through inheritance
- A ‘ / ’ marks a **default** that a subtype may override
- **Lexical rules** relate families of lexical sequences: *i-rule*s make words from lexemes, *d-rule*s make new lexemes, *pi-rule*s make words from words
- A rule states only what changes; the rule’s type and the input’s type supply everything else
- Next week: how the inherited constraints fill in a lexical rule, constant lexemes, ARG-ST and FORM, and the passive

Acknowledgments and References

- Course design and slides borrow heavily from Emily Bender's course: *Linguistics 566: Introduction to Syntax for Computational Linguistics*
  http://courses.washington.edu/ling566

- The slides on lexical types and lexical rules are adapted from Emily Bender's, with her permission; the feature structures follow Sag et al. (2003), © 2003 CSLI Publications

- This lecture complements Sag et al. (2003, Chapter 8)

References

Ivan A. Sag, Tom Wasow, and Emily Bender. 2003. *Syntactic Theory: A Formal Introduction*. CSLI Publications, Stanford, second edition.